

) To call super class constructor :

```
class A
{
    A()
    {
        super();
    }
}
class B extends A
{
    B()
    {
        super();
    }
}

B b1 = new B();
```

Object class

- * super() OR this() must be first line of any constructor.
- * We can write either super() OR this(), both we cannot write together
- * this() and super() we cannot use inside the method body, It is only for constructor.
- * From JDK 25V onwards we can write some data validation code before super() OR this() which is known as Flexible Constructor bodies.

Whenever we write a class in java and we don't write any kind of constructor to the class then the java compiler will automatically add one default no argument constructor to the class.

THE FIRST LINE OF ANY CONSTRUCTOR IS RESERVED EITHER FOR super() or this() [super of OR this of] that means first line of any constructor is used to call another constructor of either same class (this()) OR super class(super()). From Java 25V we can only perform data validation before super() OR this() [Flexible Constructor bodies]

In the first line of any constructor if we don't specify either super() or this() then the java compiler will automatically add super() to the first line of constructor.

Now the purpose of this super() [added by java compiler], to call the default constructor or No-Argument constructor of the super class.

In order to call the constructor of super class as well as same class, we have total 4 cases.

CASE 1 :

super() : Automatically added by compiler to maintain the hierarchy in the first line of the Constructor. It is used to call default OR no argument constructor of super class.

```
//Program
package com.ravi.constructor_chaning;

class A
{
    A()
    {
        IO.println("No arg constructor of A class");
    }
}
class B extends A
{
    B()
    {
        IO.println("No arg constructor of B class");
    }
}

public class ConstructorChainingDemo1 {
    public static void main(String[] args)
    {
        new B();
    }
}
```

Output :

No arg constructor of A class
No arg constructor of B class

Another Program :

```
class A
{
    A()
    {
        IO.println("No arg constructor of A class");
    }
}
class B extends A
{}

class C extends B
{
    C()
    {
        IO.println("No arg constructor of C class");
    }
}

public class InheritanceDemo
{
    public static void main(String[] args)
    {
        new C();
    }
}
```

Output :

No arg constructor of A class
No arg constructor of C class

Case 2 :

super(name) : It must be written by user. It is used to call parameterized constructor of super class.

```
package com.ravi.constructor_chaning;

class Vehicle
{
    protected String name;

    public Vehicle(String name)
    {
        super();
        this.name = name;
    }
}
class Car extends Vehicle
{
    public Car(String name)
    {
        super(name);
    }

    public String getCarName()
    {
        return this.name;
    }
}

public class ConstructorChainingDemo2
{
    public static void main(String[] args)
    {
        Car naxon = new Car("Tata Naxon");
        IO.println("Car Name is :"+naxon.getCarName());
    }
}
```

Case 3 :

this() : Must be explicitly written by user. It is used to call no argument constructor of current class.

```
package com.ravi.constructor_chaning;

class Alpha
{
    public Alpha()
    {
        IO.println("No Argument constructor of Alpha class");
    }
    public Alpha(int x)
    {
        this();
        IO.println("Parameterized constructor of Alpha class :"+x);
    }
}
class Beta extends Alpha
{
    Beta(int val)
    {
        super(val);
    }
}

public class ConstructorChainingDemo3 {
    public static void main(String[] args)
    {
        new Beta(9);
    }
}
```

Case 4 :

this("java") : Must be explicitly written by user. It is used to call parameterized constructor of current class.

```
package com.ravi.constructor_chaning;

class Alpha
{
    public Alpha()
    {
        this(15);
        IO.println("No Argument constructor of Alpha class");
    }
    public Alpha(int x)
    {
        IO.println("Parameterized constructor of Alpha class :"+x);
    }
}
class Beta extends Alpha
{
    Beta()
    {
        super();
    }
}

public class ConstructorChainingDemo3 {
    public static void main(String[] args)
    {
        new Beta();
    }
}
```

Note : In Java, Whenever we create an object then by using constructor chaining i.e. super(), the control must reach to object class first

```
class Vehicle
{
    protected String name;

    public Vehicle(String name)
    {
        super();
        this.name = name;
    }
}
class Car extends Vehicle
{
    public Car()
    {
        super(getCarName());
    }

    public String getCarName() //CE cannot call this method without object creation
    {
        return "Naxon";
    }
}
public class InheritanceDemo
{
    public static void main(String[] args)
    {
        new Car();
    }
}
```

//Program on Single level Inheritance :

```
package com.ravi.single_level_inheritance;

class Animal
{
    protected String name;
    protected double height;
    protected String color;

    public Animal(String name, double height, String color)
    {
        super();
        this.name = name;
        this.height = height;
        this.color = color;
    }
}
class Lion extends Animal
{
    public Lion(String name, double height, String color)
    {
        super(name, height, color);
    }
}
```

```
@Override
public String toString()
{
    return "Lion [name=" + name + ", height=" + height + ", color=" + color + "];"
}

public class AnimalDemo
{
    public static void main(String[] args)
    {
        Lion lion = new Lion("Sibma", 5.2, "White");
        IO.println(lion);
    }
}
```

Single Level Inheritance :

PartTimeEmp
number
name
salary

FullTimeEmp
dept
desi

```
package com.ravi.single_level_inheritance;

class TempEmp
{
    protected int id;
    protected String name;
    protected double salary;

    public TempEmp(int id, String name, double salary)
    {
        super();
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
}

class PEMP extends TempEmp
{
    protected String department;
    protected String designation;

    public PEMP(int id, String name, double salary, String department, String designation)
    {
        super(id, name, salary);
        this.department = department;
        this.designation = designation;
    }

    @Override
    public String toString()
    {
        return "PEMP [id=" + id + ", name=" + name + ", salary=" + salary + ", department=" + department + ", designation=" + designation + "];"
    }
}

public class EmployeeDemo {
    public static void main(String[] args)
    {
        PEMP p = new PEMP(101, "Scott", 90000, "IT", "Developer");
        IO.println(p);
    }
}
```

//Program on Hierarchical Inheritance :

```
package com.ravi.hierarchical;

class Employee
{
    protected double salary;
    public Employee(double salary)
    {
        super();
        this.salary = salary;
    }
}
class Developer extends Employee
{
    protected String role;

    public Developer(double salary, String role)
    {
        super(salary);
        this.role = role;
    }

    @Override
    public String toString()
    {
        return "Developer [salary=" + salary + ", role=" + role + "];"
    }
}
class Tester extends Employee
{
    protected String role;

    public Tester(double salary, String role)
    {
        super(salary);
        this.role = role;
    }

    @Override
    public String toString()
    {
        return "Tester [salary=" + salary + ", role=" + role + "];"
    }
}

public class HierarchicalDemo
{
    public static void main(String[] args)
    {
        Developer d1 = new Developer(90000, "Developer");
        IO.println(d1);

        Tester t1 = new Tester(30000, "Tester");
        IO.println(t1);
    }
}
```